JavaScript

Basics

```
var a = 1; //number
var b = '1'; //string

a == b ? true : false; // true
a === b ? true : false; // false


var x = null;
x ? true : false; // false

var x;
x ? true : false; // false

var x = '';
x ? true : false; // false

var x = 0;
x ? true : false; // false


var x = '';
y = x || 'Max';
```

Data Types

- String
- Array
- Boolean
- Date
- Function
- Object
- Number
- RegEx

```
var cars;               // Value is undefined
var person = null;      // Value is null
```

Object Creation Pattern

Object Literal

```
var User = {
    name: 'foo',
    say: function(){
        return 'Hello ' + this.name;
    }
};

User.name = 'Max';
console.log(User.say());
```

Constructor Function

```
var User = function(name){
    //!!! var this = {};

    ////private
    var _defaultName = 'Anonymous';

    //public
    this.name = name || _defaultName;

    this.say = function(){
        return 'Hello ' + this.name;
    };

    //!!! return this;
};

var user1 = new User();
user1.name = 'Max';
console.log(user1.say());
```

Constructor Function

```
//Constructor Function
var User = function(){
    var self = this;

    self.name = ''; // (this.name)
    self.say = function(){
        return 'Hello ' + self.name;
    };
    return self;
};
var user1 = new User();
user1.name = 'Max';
console.log(user1.say());

user1 instanceof User // true
```

Naming Convention

userFunction -> UserFunction

Return Object Literal

```
var Car = function(){
 var _wheels = 4; //private

        //public
 return {
  getWheels: function(){
   return _wheels;
  }
 };
};


var car1 = Car();
car1.getWheels();
```

Prototype

```
var Person = function(name){
    this.name = name;
};

Person.prototype.show = function(){
    return 'Hello ' + this.name;
};

var max = new Person('Max');
max.show();
```

If you want reusing it, add it to the prototype!

"Prefer object composition to class inheritence.

Gang of 4

Borrowing methods

```
otherObject.doStuff.call(myObject, param1, p1, p2);
otherObject.doStuff.apply(myObject, [param1, p1, p2]);
```

Mixins

Inheritence by copy properties

```
Object.prototype.mixin = function(){
 var args, prop;

 for(var arg = 0; arg < arguments.length; arg++){
  for(prop in arguments[arg]){
   this[prop] = arguments[arg][prop];
  }
 }

 return this;
};

var User = function(email){
 this.email = email;
};

var user1 = new User('info@domain.com');
user1.mixin({
 name: 'Max',
 age: 27
});
```

Classic Inheritance

```
// The inherits method is similar to Java's extends.
// The uber method is similar to Java's super.
function inherit(C,P){
 var F = function(){};
 F.prototype = P.prototype;
 C.prototype = new F();
 C.uber = P.prototype;
 C.prototype.constructor = C;
}

function Parent(){
 this.name = 'Max';
}
Parent.prototype.show = function(){
 return 'Hello ' + this.name;
};

function Child(){}

inherit(Child, Parent);

var child1 = new Child();
console.log(child1);
```

Prototypal Inheritance

```
function object(o){
 function F(){};
 F.prototype = o;
 return new F();
}

var parent = { a: 1 };
var child = object(parent);
console.log(child.a);

child.hasOwnProperty('a');
```

Functions

Functions are objects

```
var foo = function(){
};

console.log(typeof foo);
```

Immediate Functions

```
(function(){
    console.log('foo');
})();

(function(p1){
    console.log(p1);
})('foo');
```

Callbacks

```
var foo = function(a, fn){
    fn(null, a);
};

foo('bar', function(err, data){
    console.log(err);
    console.log(data);
});
```

High Order Functions

```
var add = function(x){
    return function(y){
        return x + y;
    };
};

var add5 = add(5);
var sum = add5(5);

var init = function(count){
    var c = count || 0;
    return function(){
        return c++;
    };
};

var next = init();
next();
next();
```

Self Overwriting Function

```
function next(){
    var count = 1;
    next = function(){
        return ++count;
    };
    return count;
}

next();
next();
```

Lazy Function Definition

```
function lazy(){
    var result = 1 + 1;
    lazy = function(){
        return result;
    };
    return lazy;
}

lazy();
lazy();
```

Function Object Properties

```
function memoized(param, fn){
    if(! memoized.cache) memoized.cache = {};
    if(! memoized.cache[param]){
        memoized.cache[param] = fn();
    }
    return memoized.cache[param];
}

var foo = function(){
    console.log('run');
    return 1 + 1;
};

memoized('foo', foo);
memoized('foo', foo);
memoized('foo', foo);
```

Funtion Arguments

```
var foo = function(){
    var args = Array.prototype.slice.call(arguments);
    return args;
};

foo('a', 'b');
```

Closure

```
function outer() {
    var x = 5;
    function inner() {
        console.log(x); //5
    }
    inner();
}
```

Scope

```
function outer() {
    var x = 5;
    function inner() {
        console.log(x); //5
        var y = 10;
    }
    inner();
    console.log(y); //undefined
}
```

Chaining

```
var calc = {
    value: 0,
    increment: function(){
        this.value++;
        return this;
    },
    add: function(x){
        this.value += x;
        return this;
    },
    show: function(){
        console.log(this.value);
    }
};

calc.add(5).increment().add(5).show();
```

Filter / Map / Reduce

```
var cart = [
    { name: 'TV', price: 127.92, amount: 1 },
    { name: 'Free', price: 0, amount: 1 },
    { name: 'SmartPhone', price: 65.20, amount: 0 },
    { name: 'Book', price: 7.99, amount: 1 },
    { name: 'Discount', price: -10.00 }
];

cart.filter(function(itm){
    return itm.price !== 0;
}).map(function(itm){
    return itm.price * itm.amount || 1;
}).reduce(function(total, price){
    total += price;
    return total;
}, 0);
```

Configuration Objects

```
var foo = function(name, age, gender, email){
};

var config = {
    name: '',
    age: 0,
    gender: 'M',
    email: ''
};

var bar = function(config){
};
```

Module Pattern

```
var MyModule = function(){
 var a = 1;

 return {
  get: function(){
   return a;
  }
 };
}();

MyModule.get();
```

Factory

```
var Polygon = function(){};

Polygon.Triangle = function(){};
Polygon.Circle = function(){};

Polygon.factory = function(name){
 if(typeof Polygon[name] === 'function'){
  return new Polygon[name];
 } else {
  throw new Error('Service type not found.');
 }
};

var foo = Polygon.factory('Triangle');
var bar = Polygon.factory('Circle');
```